

## Chapter 1. Introduction of DBMS (Database Management System)

A Database Management System (DBMS) is a software solution designed to efficiently manage, organize, and retrieve data in a structured manner. It serves as a critical component in modern computing, enabling organizations to store, manipulate, and secure their data effectively. From small applications to enterprise systems, DBMS plays a vital role in supporting data-driven decision-making and operational efficiency.

### 1. What is a DBMS?

A DBMS is a system that allows users to create, modify, and query databases while ensuring data integrity, security, and efficient data access. Unlike traditional file systems, DBMS **minimizes data redundancy, prevents inconsistencies, and simplifies data management** with features like concurrent access and backup mechanisms. It organizes data into tables, views, schemas, and reports, providing a structured approach to data management.

#### Example:

A university database can store and manage student information, faculty records, and administrative data, allowing seamless retrieval, insertion, and deletion of information as required.

### Key Features of DBMS

1. **Data Modelling:** Tools to create and modify data models, defining the structure and relationships within the database.
2. **Data Storage and Retrieval:** Efficient mechanisms for storing data and executing queries to retrieve it quickly.
3. **Concurrency Control:** Ensures multiple users can access the database simultaneously without conflicts.
4. **Data Integrity and Security:** Enforces rules to maintain accurate and secure data, including access controls and encryption.
5. **Backup and Recovery:** Protects data with regular backups and enables recovery in case of system failures.

### 2. Types of DBMS

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements, and application needs. The most common types are as follows:

#### a. Relational Database Management System (RDBMS)

RDBMS organizes data into tables (relations) composed of rows and columns. It uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables. Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

**Examples:** MySQL, Oracle, Microsoft SQL Server and Postgre SQL.

#### b. NoSQL DBMS

NoSQL systems are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive. They store data in various non-relational formats, such

as key-value pairs, documents, graphs, or columns. These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

**Examples:** MongoDB, Cassandra, DynamoDB and Redis.

### c. Object-Oriented DBMS (OODBMS)

OODBMS integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects. This approach supports complex data types and relationships, making it ideal for applications requiring advanced data modelling and real-world simulations.

**Examples:** ObjectDB, db4o.

## 3. Database Languages

Database languages are specialized sets of commands and instructions used to define, manipulate, and control data within a database. Each language type plays a distinct role in database management, ensuring efficient storage, retrieval, and security of data. The primary database languages include:

### a. Data Definition Language (DDL)

DDL is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function, and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT:** add comments to the data dictionary
- **RENAME:** rename an object

### b. Data Manipulation Language (DML)

DML focuses on manipulating the data stored in the database, enabling users to retrieve, add, update, and delete data.

- **SELECT:** retrieve data from a database
- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table
- **MERGE:** UPSERT operation (insert or update)
- **CALL:** call a PL/SQL or Java subprogram
- **EXPLAIN PLAN:** interpretation of the data access path
- **LOCK TABLE:** concurrency Control

### c. Data Control Language (DCL)

DCL commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT:** Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE:** Removes previously granted permissions from a user.

#### **d. Transaction Control Language (TCL)**

TCL commands oversee transactional data to maintain consistency, reliability, and atomicity.

- **ROLLBACK**: Undoes changes made during a transaction.
- **COMMIT**: Saves all changes made during a transaction.
- **SAVEPOINT**: Sets a point within a transaction to which one can later roll back.

#### **e. Data Query Language (DQL)**

DQL is a subset of DML, specifically focused on data retrieval.

- **SELECT**: The primary DQL command, used to query data from the database without altering its structure or contents.

### **4. Why we Shift from File System to DBMS**

Before the advent of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve, and update files as needed, it came with numerous challenges.

A typical example can be seen in a file-based university management system, where data was stored in separate sections such as Departments, Academics, Results, Accounts, and Hostels. Certain information like student names and phone numbers was repeated across multiple files, leading to the following issues: *(Drawback of File Systems)*

#### **a. Redundancy of data**

When the same data exists in multiple places, any update must be manually repeated everywhere. For instance, if a student changes their phone number, it must be updated across all sections. Failure to do so leads to unnecessary duplication and wasted storage.

#### **b. Inconsistency of Data**

Data is said to be inconsistent if multiple copies of the same data do not match each other. If the Phone number is different in Accounts Section and Academics Section, it will be inconsistent. Inconsistency may be because of typing errors or not updating all copies of the same data.

#### **c. Complex Data Access**

A user should know the exact location of the file to access data, so the process is very cumbersome and tedious. If the user wants to search the student hostel allotment number of a student from 10000 unsorted students' records, how difficult it can be.

#### **d. Lack of Security**

File systems provided limited control over who could access certain data. A student who gained access to a file with grades might easily alter it without proper authorization, compromising data integrity.

#### **e. No Concurrent Access**

File systems were not designed for multiple users working at the same time. If one user was editing a file, others had to wait, which hindered collaboration and slowed down workflows.

#### f. No Backup and Recovery

File systems lacked built-in mechanisms for creating backups or recovering data after a loss. If a file was accidentally deleted or corrupted, there was no easy way to restore it, potentially causing permanent data loss.

### 5. What are the advantages and disadvantages of DBMS?

#### a. Advantages of DBMS.

1. **Data organization:** A DBMS allows for the organization and storage of data in a structured manner, making it easy to retrieve and query the data as needed.
2. **Data integrity:** A DBMS provides mechanisms for enforcing data integrity constraints, such as constraints on the values of data and access controls that restrict who can access the data.
3. **Concurrent access:** A DBMS provides mechanisms for controlling concurrent access to the database, to ensure that multiple users can access the data without conflicting with each other.
4. **Data security:** A DBMS provides tools for managing the security of the data, such as controlling access to the data and encrypting sensitive data.
5. **Backup and recovery:** A DBMS provides mechanisms for backing up and recovering the data in the event of a system failure.
6. **Data sharing:** A DBMS allows multiple users to access and share the same data, which can be useful in a collaborative work environment.

#### b. Disadvantages of DBMS

1. **Complexity:** DBMS can be complex to set up and maintain, requiring specialized knowledge and skills.
2. **Performance overhead:** The use of a DBMS can add overhead to the performance of an application, especially in cases where high levels of concurrency are required.
3. **Scalability:** The use of a DBMS can limit the scalability of an application, since it requires the use of locking and other synchronization mechanisms to ensure data consistency.
4. **Cost:** The cost of purchasing, maintaining and upgrading a DBMS can be high, especially for large or complex systems.
5. **Limited Use Cases:** Not all use cases are suitable for a DBMS, some solutions don't need high reliability, consistency or security and may be better served by other types of data storage.

#### c. Applications of DBMS

1. **Enterprise Information:** Sales, accounting, human resources, Manufacturing, online retailers.
2. **Banking and Finance Sector:** Banks maintaining the customer details, accounts, loans, banking transactions, credit card transactions. Finance: Storing the information about sales and holdings, purchasing of financial stocks and bonds.
3. **University:** Maintaining the information about student course enrolled information, student grades, staff roles.
4. **Airlines:** Reservations and schedules.
5. **Telecommunications:** Prepaid, postpaid bills maintenance.

\*\*\*\*\*

## Different Types of Database Users

A Database User is defined as a person who interacts with data daily for updating, reading, and modifying the given data. Database users can access and retrieve data from the database through the Database Management System (DBMS) applications and interfaces.

### Types of Database Users

Database users are categorized based on their interaction with the database. There are seven types of database users in DBMS. Below mentioned are the types of database users:

#### 1. Database Administrator (DBA)

A Database Administrator (DBA) defines the schema and manages all three levels of the database architecture. They create user accounts, control access, and ensure data security by authorizing only trusted users. DBAs are also responsible for handling security breaches and optimizing system performance.

- DBA also monitors the recovery and backup and provides technical support.
- The DBA has a DBA account in the DBMS which is called a system or superuser account.
- DBA repairs damage caused due to hardware and/or software failures.
- DBA is the one having privileges to perform DCL (Data Control Language) operations such as GRANT and REVOKE, to allow/restrict a particular user from accessing the database.

#### Role of Database Administrator (DBA)

DBA is responsible for managing the database system. The DBA performs critical tasks such as:

- **Database design and architecture.**
- **Security management:** Implementing role-based access control (RBAC), encryption, and ensuring strong authentication measures such as multi-factor authentication (MFA).
- **Backup and recovery:** Regularly creating backups and preparing recovery plans in case of data loss.
- **Performance tuning:** Optimizing database performance, including query optimization, indexing, and resource management to ensure the DBMS runs efficiently

#### 2. Naive / Parametric End Users

Parametric End Users are the unsophisticated who lacks DBMS knowledge but they frequently use the database applications in their daily life to get the desired results. For example, Railway's ticket booking users, Clerks in any bank are naive user because they lack DBMS knowledge but they still use the database.

#### 3. A System Analyst

A system Analyst is a user who analyzes the requirements of parametric end users. They check whether all the requirements of end users are satisfied.

#### 4. Sophisticated Users

Sophisticated users like engineers or analysts interact directly with the database using SQL queries. They don't write full application code but use the query processor to access and manipulate data. These users develop custom database applications based on their specific needs.

## **5. Database Designers**

Database Designers create the structure of a database, including tables, views, indexes, triggers, and constraints. They decide what data to store and how it should be related based on user requirements. Their design is finalized before the database is built or populated with data.

## **6. Application Programmers**

Application Programmers, also known as System Analysts or Software Engineers, write backend code for application programs. They use languages like C, Visual Basic, COBOL, etc., to design, test, and maintain programs. These programs, called "canned transactions," help naive users interact with the database efficiently.

## **7. Casual Users / Temporary Users**

Casual Users are the users who occasionally use/access the database but each time when they access the database they require the new information, for example, Middle or higher level manager.

## **8. Specialized users**

Specialized users are sophisticated users who write specialized database application that does not fit into the traditional data-processing framework. Among these applications are computer aided-design systems, knowledge-base and expert systems etc.

## **Q. Explain structure of the Database Management System**

A Database Management System (DBMS) is software that allows users to define, store, maintain, and manage data in a structured and efficient manner. It acts as an intermediary between data and users, allowing disparate data from different applications to be managed. A DBMS simplifies the complexity of data processing by providing tools to organize data, ensure its integrity, and prevent unauthorized access or loss of data.

In today's data-driven world, DBMS are essential for applications such as banking systems, e-commerce platforms, education, and medical systems. They not only store and manage large amounts of data, but also provide functionality that provides performance, security, and scalability for multiple users with multiple access levels.

**It also allows access to data stored in a database and provides an easy and effective method of:**

- Defining the information.
- Storing the information.
- Manipulating the information.
- Protecting the information from system crashes or data theft.
- Differentiating access permissions for different users.

## **Understanding Data Theft in DBMS**

Data theft means the illegal extraction or manipulation of sensitive information stored in databases, servers, and other storage systems. This is further defined, in DBMS, as improper access to confidential or sensitive data by unauthorized persons.

This may include information such as personal data, financial records, intellectual property, or trade secrets. As digital data storage has grown, so has the threat of data theft; it is now a primary priority concern with serious impacts on organizations worldwide.

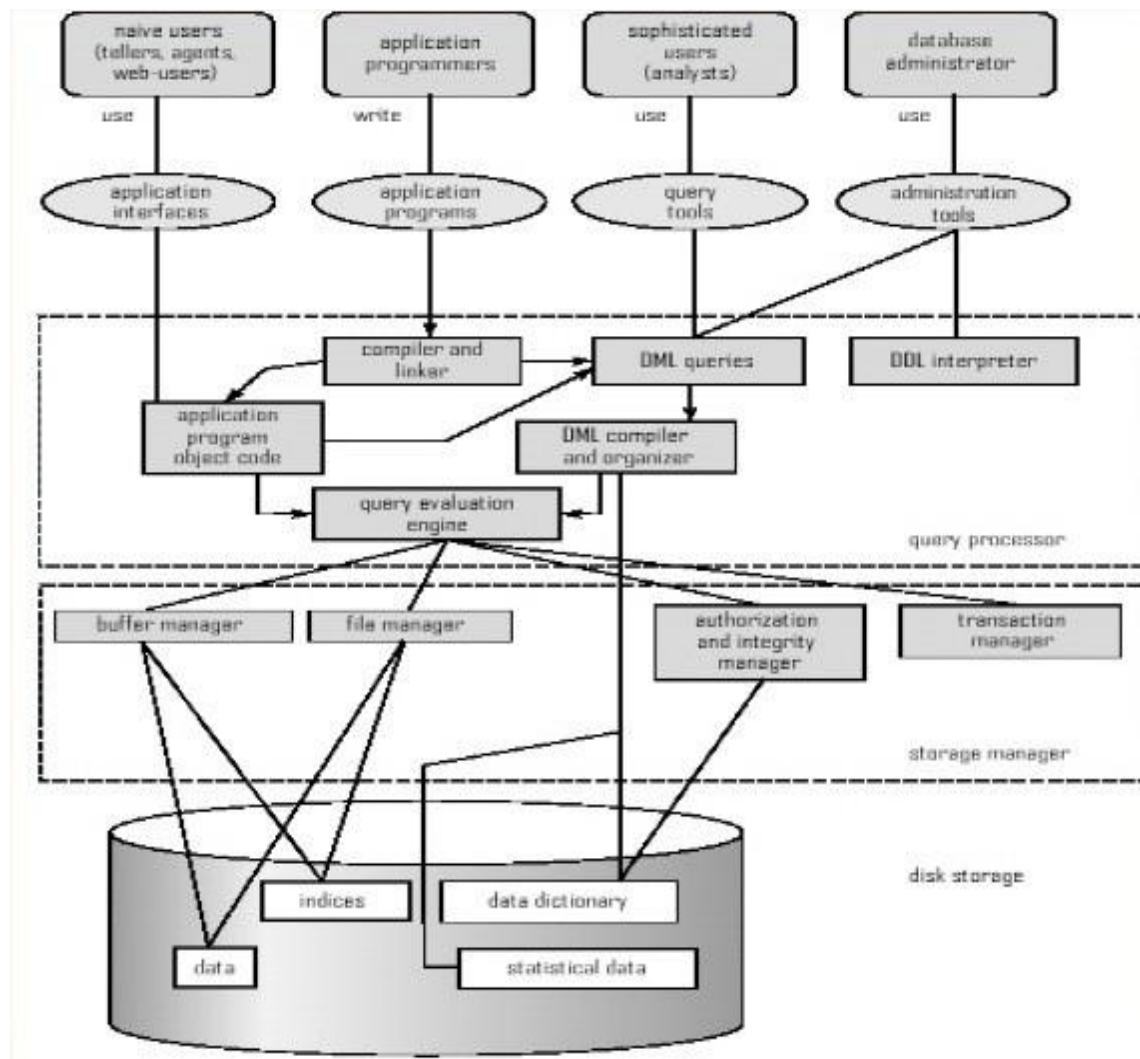
**Data theft can be carried out by, among others:**

- **Hacking and exploiting:** Attackers can use DBMS security gaps to access unauthorized sensitive data.
- **Insider threats:** Employees or contractors compromise privileged access to information.
- **Phishing and social engineering:** These are techniques that will trick the authorized user into revealing the login credentials to enable intrusion.
- **Malware and ransomware attacks:** These are malware that make database security vulnerable to attack, thus giving access to attackers to steal data or lock down data until some amount of ransom is paid.

**Data theft prevention** is not only an issue in sensitive information matters but also for building trust between businesses and clients. Controls over access, periodic audits, real-time monitoring of activities done through the database are effective measures one could consider reducing the risk. Also, following cyber security protocols and periodic inundation of database systems will reduce most of the vulnerabilities.

## Database Architecture

- Also included as Components of database management or database structure.
- A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- Functional components of a database system can be broadly divided into,
  - Storage Manager
  - Query Processor



Database Architecture / System Structure

## 1. Disk Storage

- All data or information stored in disk storage
- Various components of disk storage
  - **Data:** Data files which stores the database itself
  - **Data directory:**
    - It contains meta data (data about data)
    - Schema of table is an example of meta data
    - A database system consults the data directory before reading and modifying the actual data.
  - **Indices:** It provides fast access to data items that holds values.
  - **Statistical data:** Stores information/statistics of data stored.

## 2. Storage Manager

- Storage manager is a program module that provide interface between,
  - Low level data stored in database and
  - Application programs and
  - Queries submitted to system.
- Storage manager is responsible for the interaction with file manager.
- Storage manager translates DML statements into low-level file system commands.
- Storage manager is responsible for
  - Storing
  - Retrieving
  - Updating data in database

## Components of Storage manager

- a) **Authorization and integrity manager**
  - Checks **authority of users** accessing the data
  - It **tests integrity and constraints of data**
- b) **Transaction manager**
  - It ensures that the database remains in a consistent state despite of failure.
  - **Concurrent execution in maintained without any conflict.**
- c) **File manager**
  - It manages **allocation of space** on disk storage
  - Information stored on disk is represented using database.
- d) **Buffer manager**
  - It is responsible for fetching data from disk storage into main memory.
  - It decides what data to cache the main memory

### 3. Query processor

- Query processor is an important part of the database system
- It helps the database system to simplify and facilitate access to data.

Various components of Query processors

#### a) DDL Interpreter

- It interprets DDL statements at a low level.
- Records the definition in the data directory.

#### b) DML Compiler

- It translates DML query statements in query language into low-level instruction.
- Query evaluation engine understands only low-level instruction.
- Query can be translated into many numbers of evaluation plans which produces same result.
- Query optimization is picking up the lowest cost evaluation plan among many alternatives. It is performed by DML compiler.

#### c) Query evaluation engine

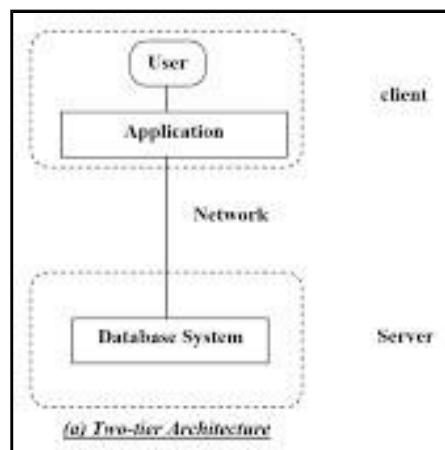
- Execute low level instruction generated by DML compiler.

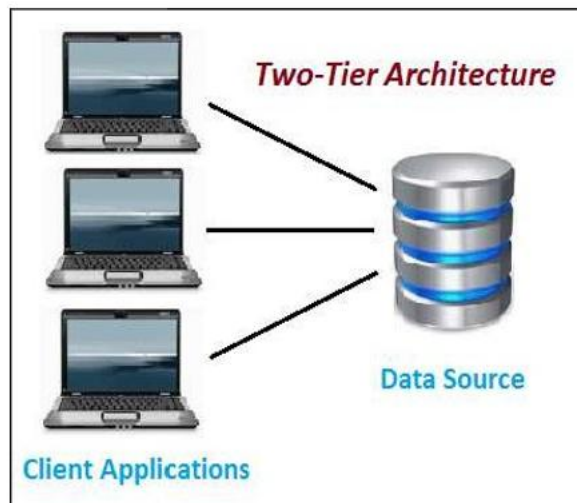
Database architecture can be of two types,

- Two Tier Architecture
- Three Tier Architecture

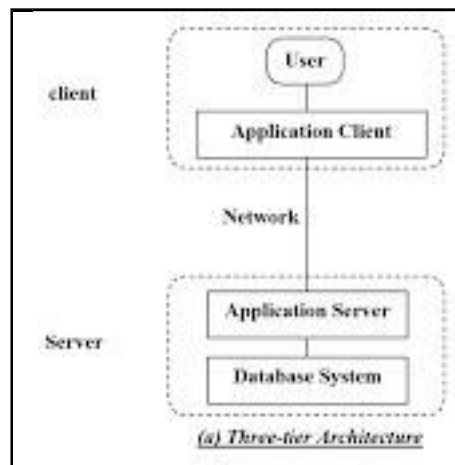
Two Tier Architecture

- Usually database application is partitioned into two parts.
- The two-tier is based on Client Server architecture.
- The two-tier architecture is like client server application.
- The direct communication takes place between client and server.
- There is no intermediate between client and server.





### Three Tier Architecture



- In Three tier architecture, client machine acts as front end and does not contain any direct database calls.
- Instead client end communicate with an application server Application server in turn communicates with a database system to access data.
- Three-tier architecture typically comprises a presentation tier, a business or data access tier, and a data tier.

Three layers in the three tier architecture are as follows:

- 1) Client layer,
- 2) Business layer,
- 3) Data layer

**1) Client layer:**

- It is also called as Presentation layer which contains UI part of our application.
- This layer is used for the design purpose where data is presented to the user or input is taken from the user.
- For example designing registration form which contains text box, label, button etc.

**2) Business layer:**

- In this layer all business logic written like validation of data, calculations, data insertion etc.
- This acts as an interface between Client layer and Data Access Layer.

- This layer is also called the intermediary layer and helps to make faster between client and data layer.
- Data layer: In this layer an actual database comes in the picture. Data Access Layer contains methods to connect with database and to perform insert, update, delete, get data from database based on our input data.

